FinTwin

An AI-Assisted Banking Database Performance Simulator and Performance Digital Twin for PostgreSQL

FINAL PROJECT REVIEW REPORT

Database Systems Project

Submitted by

Team Member	Registration Number
Arav Singh Patel	24BCE2425
Chaitanya Ghogare	24BCE2392

B.Tech Computer Science and Engineering

VIT Vellore

October 2026

Source repository: https://github.com/Arrav-SP/FinTwin

Abstract

Relational database performance depends on the workload that is applied: the mix of operations, the arrival rate and the number of concurrent clients. Measuring every configuration of interest directly is slow, so it is attractive to learn the relationship between workload and performance from a limited set of measured experiments and to use it to estimate the behaviour of configurations that have not been run. FinTwin is a prototype of such a system for a banking workload on PostgreSQL 16.

The project implements a complete measure, learn, predict and validate loop. A normalized banking schema of six tables is populated with deterministic synthetic data. A workload generator executes seven banking scenarios, built from seven operation types, against the live database at a controlled rate and concurrency, using transactions with row-level locking and ordered lock acquisition for transfers. A telemetry collector samples PostgreSQL statistics views and host resource usage once per second. A scikit-learn pipeline trains regression models that map pre-execution workload configuration to five measured outcomes, with safeguards against target leakage and warnings for out-of-range inputs. A what-if engine, a rule-based optimization advisor, a predicted-versus-actual validation engine, a REST API and a dependency-free web interface complete the system.

All components are implemented and integrated. The predictive accuracy is not yet established: the models were trained on seven usable experiments, and the single recorded validation run showed errors of 49.7% for average latency, 66.5% for P95 latency, 20.6% for throughput and 50.5% for host CPU. The report therefore presents FinTwin as a working experimental platform whose engineering is complete and whose machine learning results are preliminary, and it analyses why the present dataset cannot support accurate prediction and what is required to change that.

Keywords: database performance, PostgreSQL, transaction processing, concurrency control, workload generation, telemetry, regression, digital twin, what-if analysis.

Table of Contents

Abstract 2
1. Introduction 5
1.1 Background and motivation 5
1.2 Problem statement 5
1.3 Objectives 5
1.4 Scope and status 6
1.5 Contributions 6
2. Background 7
2.1 Performance metrics 7
2.2 Transactions, locking and deadlock 7
2.3 PostgreSQL statistics views 7
2.4 Learning performance models 7
3. System Overview and Architecture 7
3.1 The experimental loop 7
3.2 Components 8
3.3 Technology choices 9
4. Database Design 10
4.1 Conceptual model 10
4.2 Relational schema and integrity constraints 10
4.3 Normalization 11
4.4 Indexes 11
4.5 Experiment platform tables 12
4.6 Synthetic data 12
5. Transaction Processing and Concurrency Control 12
5.1 The transfer transaction 12
5.2 ACID properties 13
5.3 Deadlock avoidance by lock ordering 13
5.4 Other operations 13
6. Workload Generation 14
6.1 Scenarios and operation mix 14
6.2 Configuration 15
6.3 Pacing and concurrency 15
6.4 What the latency measurement includes 15
6.5 Reproducibility 16
7. Telemetry Collection 16
7.1 Sampling 16
7.2 Experiment summary 17
7.3 Interpreting the telemetry 17
8. Machine Learning Pipeline 17
8.1 Dataset 17
8.2 Features and targets 17
8.3 Training and model selection 18
8.4 Prediction and out-of-range warnings 18
9. What-If Analysis 19
10. Optimization Advisor 19
11. Validation Framework 20
11.1 Lifecycle 20
11.2 Error metrics 20

12. API and User Interface....21
12.1 REST API....21
12.2 Web interface....21
13. Testing and Results....22
13.1 Automated tests....22
13.2 Workload runs....23
13.3 Telemetry for one experiment....23
13.4 Model training....23
13.5 What-if analysis....24
13.6 Optimization assessment....25
13.7 Predicted versus actual....25
13.8 Benchmark runs and run-to-run variation....26
14. Discussion....27
14.1 What has been achieved....27
14.2 Why the models do not yet predict well....27
14.3 Value of the result....27
15. Limitations....28
15.1 Data and models....28
15.2 Workload realism....28
15.3 Measurement....28
15.4 Engineering....28
16. Future Work....29
17. Conclusion....29
References....30
Appendix A. Reproducing the Experiments....31

1. Introduction

1.1 Background and motivation

An online transaction processing (OLTP) database does not have a single performance figure. Latency and throughput depend on how many clients are active, how quickly requests arrive and which operations they perform. Write transactions that update the same rows contend for locks, read queries compete for buffer cache and CPU, and the response time seen by a client is a result of all of these interactions [1]. A bank is a clear example: an ordinary weekday, a salary-credit day and a festival shopping peak place very different demands on the same schema.

The usual way to answer a capacity question is to run a load test for the configuration in question and measure the result, as standard benchmarks such as TPC-C [5] and frameworks such as OLTP-Bench [4] do. This is accurate but slow, and the number of possible configurations grows quickly with every parameter. A complementary approach is to treat a set of measured experiments as training data and to learn a model of the system's performance, in the spirit of a digital twin [2] and of machine-learning-based database tuning systems such as OtterTune [3]. FinTwin explores this approach at the scale of a course project.

1.2 Problem statement

Given a PostgreSQL database with a banking schema, and a workload described by a scenario, an operation mix, a target transaction rate, a concurrency level and a duration, estimate the latency, throughput, host CPU utilisation and lock waiting that the workload will produce, without executing it; and provide a way to check any such estimate against a real execution of the same workload.

The project deliberately models the performance of the database and not the business logic of a bank. The banking domain supplies a realistic transactional schema and realistic operations; it is not connected to any real financial system and uses only synthetic data.

1.3 Objectives

- Design a normalized relational schema for a simplified bank in PostgreSQL, with integrity enforced by keys and constraints.
- Implement atomic, isolated banking operations and a workload generator that executes configurable scenarios against the live database.
- Collect database and host telemetry for every experiment and store it alongside the workload results.
- Train regression models that predict performance from workload configuration only, and expose them through a prediction interface.
- Provide what-if comparison of workload configurations using the models.
- Detect likely bottlenecks in measured experiments with transparent, rule-based checks.
- Validate predictions by executing the predicted configuration and computing the error.

- Provide a REST API and a web interface that present only measured or model-derived values.

1.4 Scope and status

Table 1 summarises the status of each objective at the time of this review. The statements in this report were checked against the source code, the stored model artefacts and the recorded result files in the repository; where a capability is partial, the table says so.

Table 1. Status of project objectives.

Objective	Status	Evidence in the repository
Relational banking schema	Implemented	`database/schema.sql`; 6 banking tables, 5 platform tables
Transactions and workload generation	Implemented	`services/transfer_service.py`, `workload/runner.py`; 7 scenarios, 7 operation types
Telemetry collection	Implemented	`telemetry/collector.py`; tables `telemetry_samples`, `experiment_summaries`
Performance prediction	Implemented; accuracy not established	`ml/` package; 5 persisted models trained on 5 of 7 usable experiments
What-if analysis	Implemented	`what_if.py`; comparison through API and CLI, sweeps through CLI
Optimization advisor	Implemented	`optimization.py`; six threshold rules, advisory only
Predicted-versus-actual validation	Implemented; one run recorded	`validation.py`; `artifacts/validation.json`
API and web interface	Implemented	`main.py` (12 REST endpoints); `frontend/` (six views)

1.5 Contributions

- An end-to-end experimental platform in which workload execution, telemetry, model training, prediction and validation share one database and one experiment identifier, so that every prediction can be traced to the measurement that tests it.
- A banking workload generator with rate pacing that is separated from latency measurement, and with deterministic operation selection from a seed.
- A leakage-safe learning pipeline: only quantities known before execution are admitted as features, the train/test split is grouped by configuration, training is refused below a minimum dataset size, and predictions carry out-of-range warnings.
- A validation workflow that records failed predictions and failed runs as failures instead of converting them into accuracy figures.
- An honest evaluation that reports where the current models do not work and explains why.

2. Background

2.1 Performance metrics

Throughput is the number of operations completed per unit time, reported here in transactions per second (TPS). Latency is the time one operation takes. Because latency distributions in database systems are skewed, the mean hides the slow requests that users notice, and percentiles are reported as well: P50 (median), P95 and P99. FinTwin computes percentiles by linear interpolation between order statistics over the per-operation latency samples of an experiment.

2.2 Transactions, locking and deadlock

A transaction is a unit of work that is atomic, consistent, isolated and durable (ACID) [1]. PostgreSQL implements isolation with multi-version concurrency control and row-level locks; `SELECT ... FOR UPDATE` locks the selected rows against concurrent updates until the transaction ends [7]. Two transactions that each hold a lock the other needs form a deadlock. One of the four necessary conditions for deadlock is circular wait, and a classical way to prevent it is to make every transaction acquire resources in the same global order [6]. FinTwin applies this technique to account rows.

2.3 PostgreSQL statistics views

PostgreSQL exposes its internal state through system views [7]. FinTwin reads `pg_stat_activity` (one row per server process, with its state and wait event), `pg_stat_database` (cumulative counters per database, including commits, rollbacks, blocks read and hit, and deadlocks) and `pg_locks` (locks held or awaited). The optional `pg_stat_statements` extension adds per-statement statistics but must be preloaded by the server; it is not enabled in the project's Docker configuration.

2.4 Learning performance models

Predicting performance from configuration is a supervised regression problem. FinTwin evaluates three standard regressors from scikit-learn [8]: ordinary least-squares linear regression, a random forest [9] and gradient-boosted regression trees [10]. Two properties of these models matter for the results in Section 13. A linear model extrapolates along a straight line outside the range of its training data, whether or not the real system does. A tree ensemble cannot extrapolate at all: outside the training range its prediction is constant. A further risk in this setting is target leakage, where information that is only available after the outcome is known enters the features and inflates apparent accuracy [11]; the pipeline is designed to exclude it.

3. System Overview and Architecture

3.1 The experimental loop

FinTwin is organised around a loop of five activities. **Generate:** a workload configuration is turned into a paced stream of banking operations. **Observe:** each operation's latency and

outcome are recorded, while a sampler records database and host statistics. **Learn:** completed experiments become rows of a training set, from which one model per target metric is fitted. **Predict:** a new configuration is passed to the models to obtain estimated metrics, with warnings when it lies outside the training data. **Validate:** the same configuration is executed, and the estimates are compared with the measurements.

3.2 Components

Figure 1 shows the components and their dependencies. The FastAPI application is a thin layer: it validates requests and delegates to the same backend modules that the command-line scripts call directly. PostgreSQL is both the system under test and the store for experiment results, which keeps predictions, measurements and validations linked by foreign keys. Table 2 lists each component with its location in the source tree.

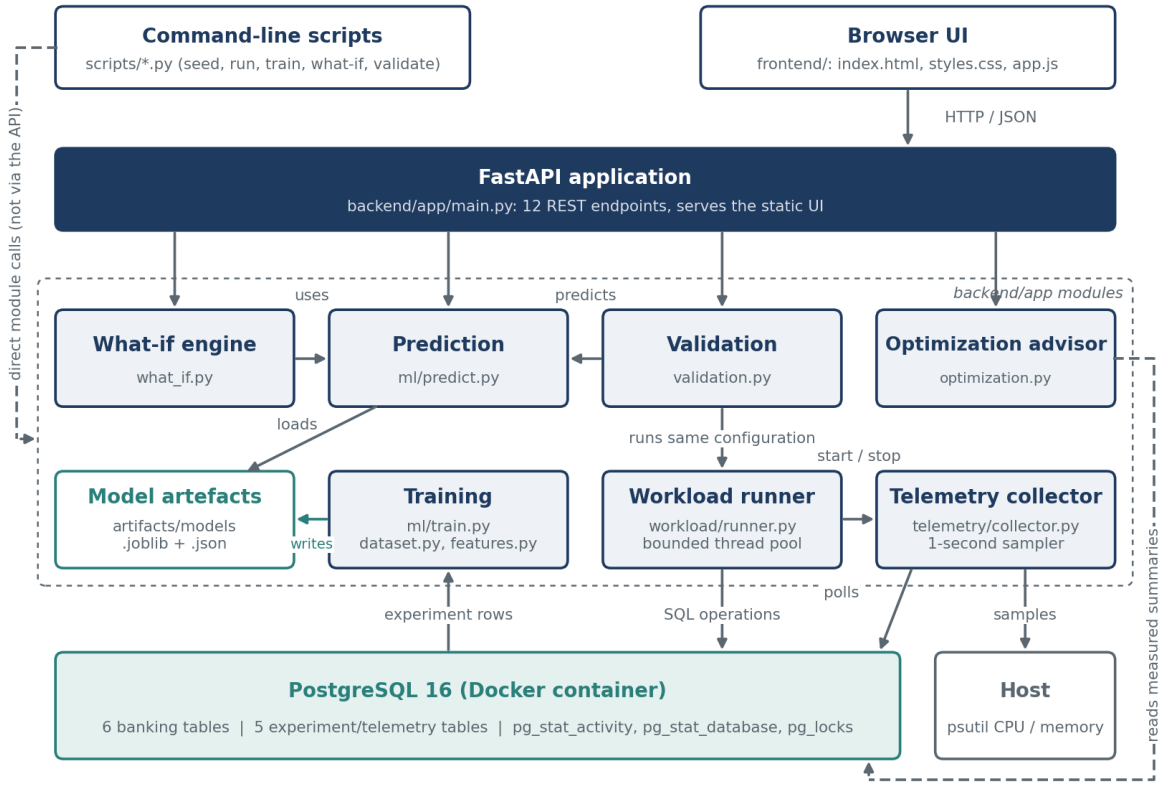


Figure 1. FinTwin architecture. Solid arrows are calls or data flow at run time; dashed arrows are paths that do not pass through the API.

Table 2. Components and their responsibilities.

Component	Source	Responsibility
Schema and seed	`database/schema.sql`, `scripts/seed_database.py`	Defines all tables, constraints and indexes; loads deterministic synthetic data (seed 42).
Transfer service	`backend/app/services/transfer_service.py`	Atomic fund transfer with ordered row locking.
Workload runner	`backend/app/workload/`	Scenario definitions, configuration validation, paced concurrent execution, per-operation event logging.
Telemetry collector	`backend/app/telemetry/`	Background sampling of PostgreSQL views and host metrics; per-experiment summary.
ML pipeline	`backend/app/ml/`	Dataset extraction, feature definition, training with model selection, prediction with range warnings.
What-if engine	`backend/app/what_if.py`	Baseline-versus-modified comparison, parameter sweeps, sensitivity.
Optimization advisor	`backend/app/optimization.py`	Threshold rules over measured experiment metrics; advisory recommendations.
Validation engine	`backend/app/validation.py`	Predict, execute, compare; error metrics and aggregate summaries.
API	`backend/app/main.py`	REST endpoints and static file serving.
Web interface	`frontend/`	Six views over the API; no framework and no build step.

3.3 Technology choices

Table 3 lists the technologies used and the reason each was chosen.

Table 3. Technology stack and the reason for each choice.

Technology	Role	Reason
PostgreSQL 16 (Docker, `postgres:16-alpine`)	System under test and result store	ACID transactions, row-level locking and detailed statistics views; a container gives a reproducible server with a named volume and a health check.
Python 3.11+	Implementation language	Required for `StrEnum`; one language for workload, API and ML.
SQLAlchemy 2.x Core with psycopg 3	Database access	SQL is written explicitly with `text()`, so the statements that are measured are exactly the statements in the source; connection pooling is provided by the engine.
FastAPI and Pydantic	REST API	Declarative request validation (ranges, enumerations) and automatic conversion of errors to HTTP status codes [12].
pandas, scikit-learn, joblib	Dataset, models, persistence	Standard, interpretable regressors; a `Pipeline` stores preprocessing with the model.
psutil	Host metrics	Portable CPU and memory readings.
HTML, CSS, JavaScript (no framework)	Web interface	Three static files served by the API; no dependency installation or build tooling.
pytest, httpx	Testing	Unit and integration tests, including API tests through the FastAPI test client.

4. Database Design

4.1 Conceptual model

The banking domain is modelled with six entities. A **branch** serves many customers and holds many accounts. A **customer** owns one or more accounts. An **account** participates in many transactions, as the source, the destination or both roles of different transactions. A **merchant** receives card or point-of-sale payments. A **transaction** records one movement of money and references up to two accounts and, for payments, a merchant. A **loan** belongs to a customer, is linked to one of the customer's accounts and is managed by a branch. Figure 2 shows the resulting relational schema together with the five tables that the platform itself writes.

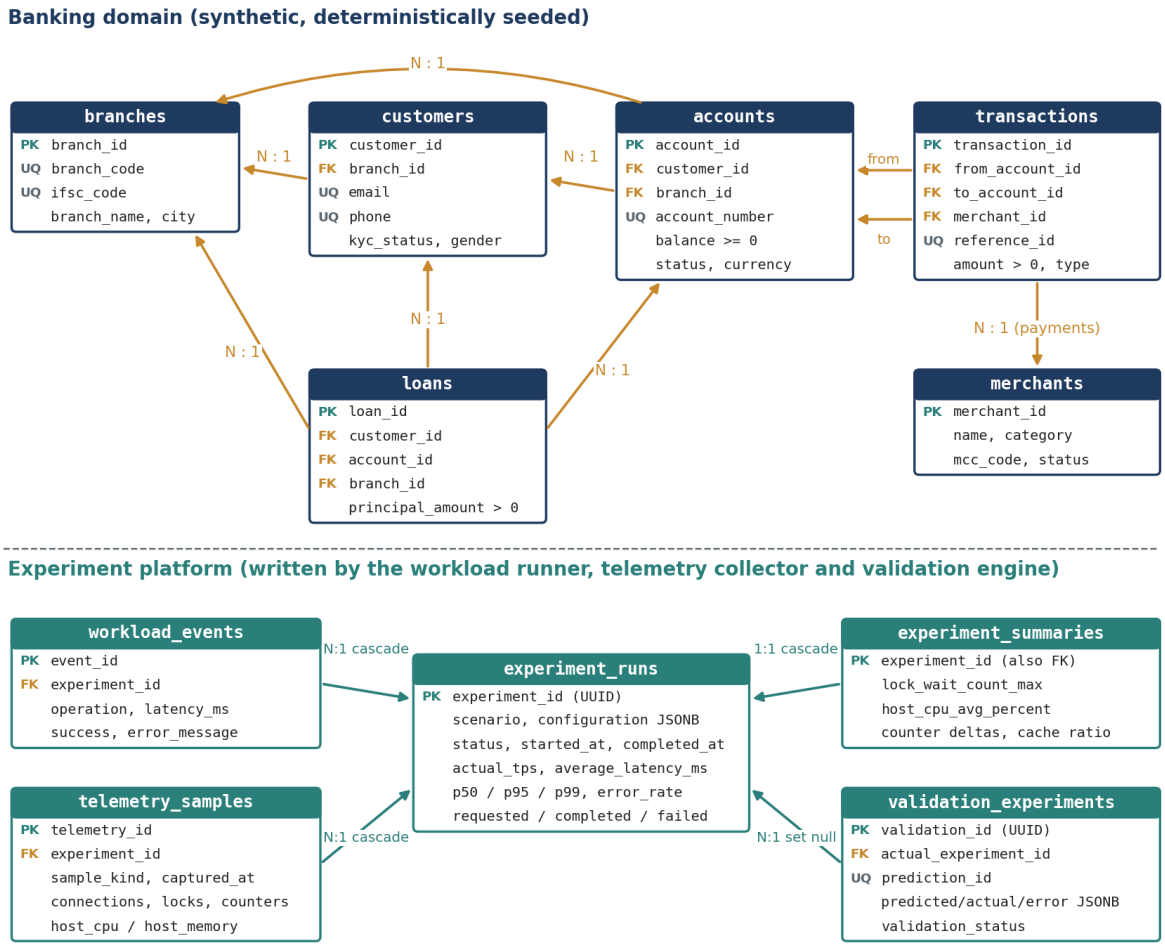


Figure 2. Relational schema defined in `database/schema.sql`. Arrows point from the referencing table to the referenced table. Only key and representative columns are shown.

4.2 Relational schema and integrity constraints

Every table has a single-column surrogate primary key (`BIGINT GENERATED BY DEFAULT AS IDENTITY` for banking tables, `UUID` for experiments and validations). Table 4 lists the keys and

the main declarative constraints. Constraints are enforced by the database, so an invalid row is rejected regardless of which application code attempts to insert it.

Table 4. Banking tables: keys and principal constraints.

Table	Seeded rows	Foreign keys	Principal constraints
branches	100	none	branch_code and ifsc_code unique and non-empty
customers	1,000	branch_id	email and phone unique; gender and kyc_status restricted to enumerated values
accounts	1,500	customer_id, branch_id	account_number unique; balance >= 0; currency = 'INR'; a closed account must have a closed_date not earlier than open_date
merchants	100	none	status restricted to ACTIVE or INACTIVE
transactions	5,000	from_account_id, to_account_id, merchant_id	amount > 0; reference_id unique; source and destination must differ; shape constraint by type (below)
loans	200	customer_id, account_id, branch_id	principal_amount > 0; interest_rate >= 0; tenure_months > 0; end_date > start_date

The constraint transactions_shape_ck makes the meaning of each transaction type explicit in the schema: a TRANSFER must have both a source and a destination account, a DEPOSIT must have a destination, a WITHDRAWAL must have a source, and a PAYMENT must have a source account and a merchant. A structurally invalid transaction row therefore cannot be stored.

4.3 Normalization

All attributes are atomic and there are no repeating groups, so the tables are in first normal form. Every primary key is a single attribute, so no partial dependency on part of a key is possible and the tables are in second normal form. With respect to the functional dependencies that the design intends, in which each primary key determines the other attributes of its table, non-key attributes do not determine one another, and the design targets third normal form. Customer data is stored once and referenced from accounts; branch data is stored once and referenced from customers, accounts and loans; merchant data is stored once and referenced from payments.

Two points deserve an honest note because an examiner may raise them. First, loans stores both account_id and customer_id. If a loan's customer is always the owner of the linked account, then customer_id is determined by account_id, which is a transitive dependency; the schema keeps both columns for direct joins and does not enforce that they agree. Second, loans stores tenure_months together with start_date and end_date without a constraint relating them, and the seed data assigns them independently. Both are acceptable in a synthetic performance test bed but would be tightened in a production design.

4.4 Indexes

PostgreSQL creates an index for every primary key and unique constraint. The schema adds fifteen explicit indexes: nine on the banking tables (foreign keys of accounts, transactions

and `loans`, plus `transactions.transaction_time`), two on `workload_events` (experiment and time), one composite index on `telemetry_samples (experiment_id, captured_at)` for time-ordered retrieval of an experiment's samples, and three on `validation_experiments` (status, scenario and linked experiment). The indexes on `from_account_id` and `to_account_id` support the recent-transactions query of the workload.

4.5 Experiment platform tables

- `experiment_runs`: one row per workload execution, with the scenario, the full configuration as JSONB, status, operation counts, measured TPS and latency statistics.
- `workload_events`: one row per executed operation, with its type, latency, success flag and error message.
- `telemetry_samples`: timestamped snapshots of database and host metrics, marked INITIAL, SAMPLE or FINAL.
- `experiment_summaries`: one row per experiment with aggregated telemetry (maxima, counter deltas, averages).
- `validation_experiments`: one row per validation, holding the configuration, predicted metrics, actual metrics, per-metric errors and a status.

Events, samples and summaries reference `experiment_runs` with `ON DELETE CASCADE`. A validation references its experiment with `ON DELETE SET NULL`, so a validation record survives deletion of the run it measured. The schema file is idempotent (`CREATE ... IF NOT EXISTS`) and is applied by `scripts/initialize_database.py`.

4.6 Synthetic data

`scripts/seed_database.py` truncates the six banking tables and reloads them inside one transaction, using a pseudo-random generator with a fixed seed of 42, so the same data is produced on every run. All seeded accounts are ACTIVE with balances between 500 and 199,999, and all seeded customers are KYC-verified. The seeded transaction history is independent of the seeded balances; it provides rows for read queries and is not a ledger that reconciles to the balances.

5. Transaction Processing and Concurrency Control

5.1 The transfer transaction

`transfer_funds(source, destination, amount)` is the central write operation. It rejects non-positive amounts and identical accounts before touching the database, and then performs the following steps inside a single transaction opened with `engine.begin()`:

- **Lock.** Both account rows are selected with `SELECT ... WHERE account_id IN (:first_id, :second_id) ORDER BY account_id FOR UPDATE`.
- **Validate.** Both accounts must exist and be ACTIVE, use the same currency, and the source balance must cover the amount. Any failure raises `TransferError`.
- **Update.** The source balance is decreased and the destination balance increased.

- **Record.** A TRANSFER row with a unique reference identifier is inserted into `transactions`.
- **Commit.** Leaving the `engine.begin()` block commits; an exception at any step rolls back every change.

5.2 ACID properties

Table 5 relates each ACID property to the mechanism that provides it in the transfer transaction.

Table 5. How the implementation provides each ACID property.

Property	Mechanism in FinTwin
Atomicity	The debit, the credit and the transaction record are executed in one database transaction; an exception rolls back all three. An integration test confirms that an insufficient-balance transfer leaves both balances unchanged.
Consistency	The application checks the balance before updating, and the database independently enforces `balance >= 0`, positive amounts and the transaction shape constraint.
Isolation	`FOR UPDATE` row locks prevent two concurrent transactions from reading and updating the same balance at once. The isolation level is PostgreSQL's default, READ COMMITTED; the code does not change it.
Durability	Provided by PostgreSQL's write-ahead log once the commit returns. The data directory is on a named Docker volume and survives container restarts.

5.3 Deadlock avoidance by lock ordering

Consider two simultaneous transfers, from account A to B and from B to A. If each transaction locked its source first, the first could hold A and wait for B while the second holds B and waits for A. FinTwin avoids this by requesting both rows in one statement ordered by `account_id`, so every transfer acquires the lower-numbered account first, whatever the direction of the transfer. With a single global acquisition order, a cycle of transactions waiting on account rows cannot form [6].

The other write operations of the workload (deposit, withdrawal and payment) each lock exactly one account row, so they cannot take part in a cycle over account rows either. The claim made here is limited to what the code establishes: ordered locking removes the circular-wait condition for account row locks in the implemented operations. It has not been stress-tested. No automated test runs opposing transfers concurrently, and the recorded experiments, which reported a deadlock count of zero, were run at loads too light to produce lock contention (Section 14).

5.4 Other operations

Deposits, withdrawals and merchant payments follow the same pattern on a single account: lock the row with `FOR UPDATE`, check the balance where money leaves the account, update it, and insert the corresponding transaction row, all in one transaction. The two read operations (balance check and recent transactions) and the loan aggregation query run without explicit locks.

6. Workload Generation

6.1 Scenarios and operation mix

A scenario is a probability distribution over seven operation types. Seven scenarios are defined in `workload/config.py`; their default mixes are shown in Table 6 and Figure 3. A configuration may also supply its own mix, which must contain only positive weights that sum to one. Table 7 describes the database work performed by each operation type.

Table 6. Default operation mix of each scenario (percentage of operations).

Scenario	Balance check	Recent txns	Transfer	Deposit	Withdrawal	Payment	Loan aggr.
NORMAL_DAY	35	20	20	10	10	5	0
SALARY_DAY	25	10	20	35	0	10	0
MONTH_END	20	20	15	0	0	15	30
FESTIVAL_SPIKE	25	15	10	10	0	40	0
CARD_PAYMENT_SPIKE	20	15	5	0	0	60	0
LOAN_PROCESSING	20	5	10	10	0	0	55
HIGH_CONCURRENCY_ TRANSFER	15	10	70	0	0	5	0

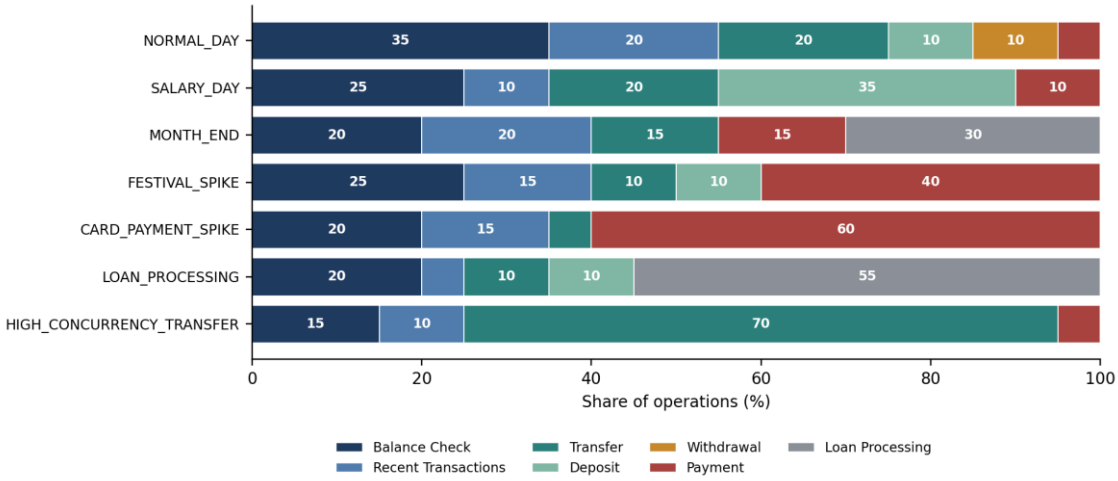


Figure 3. Operation mix of the seven scenarios, generated from `DEFAULT_MIXES` in the source.

Table 7. Operation types and the SQL work each performs.

Operation	Kind	Database work
BALANCE_CHECK	Read	Select the balance of one account by primary key.
RECENT_ TRANSACTIONS	Read	Select the 20 most recent transactions in which an account is source or destination.
TRANSFER	Write	Ordered lock of two accounts, two balance updates, one insert (Section 5.1).
DEPOSIT	Write	Lock one account, increase its balance, insert a DEPOSIT row.
WITHDRAWAL	Write	Lock one account, check and decrease its balance, insert a WITHDRAWAL row.

Operation	Kind	Database work
PAYMENT	Write	Choose a random active merchant, lock one account, check and decrease its balance, insert a PAYMENT row.
LOAN_PROCESSING	Read (aggregate)	Count loans and sum principal grouped by branch over the whole `loans` table.

6.2 Configuration

A workload is described by an immutable `WorkloadConfig`: scenario, concurrency (default 4), target TPS (default 10), duration in seconds (default 10), an optional operation mix, a random seed (default 42) and a burstiness value between 0 and 1. The constructor rejects non-positive concurrency, rate or duration and invalid mixes. The burstiness value is validated, stored with the experiment and used as a model feature, but the runner does not yet use it to shape arrivals; every recorded experiment has burstiness 0.

6.3 Pacing and concurrency

The runner computes the number of operations as the target TPS multiplied by the duration, and submits them to a thread pool whose size is the configured concurrency. Operation *i* is scheduled at offset *i* divided by the target TPS from the start of the run; the submitting thread sleeps until that offset is reached:

```
target_offset = index / self.config.target_tps
remaining = target_offset - (perf_counter() - started_clock)
if remaining > 0:
    time.sleep(remaining)
```

This is an open-loop schedule: arrivals do not wait for earlier operations to complete. The target TPS is therefore an input, whereas the **actual TPS** is a measurement, computed as completed operations divided by the wall-clock time until the last operation finishes. If the database cannot keep up, operations queue in the pool and the actual TPS falls below the target.

Each worker obtains connections from the SQLAlchemy engine's pool. The engine is created with default pool settings (documented defaults: five pooled connections and up to ten overflow connections [13]), so at most fifteen connections are open at once. A concurrency setting above fifteen would make workers wait for a connection rather than increase concurrency inside the database. All recorded experiments used twelve workers or fewer.

6.4 What the latency measurement includes

The latency of an operation is measured inside the worker thread with `time.perf_counter()`, from the moment the worker starts the operation until it finishes or raises an exception. Three consequences follow and should be kept in mind when reading the results:

- The pacing sleep and any time an operation spends waiting in the thread pool queue are **excluded**. Under overload, the reported latency is service time, not the response time an end user would see.
- Each operation first selects the identifiers of up to 1,000 active accounts (ordered by `account_id`) and samples from them. This lookup query is **included** in the measured

latency, and it means that operations are spread uniformly over the first 1,000 of the 1,500 accounts.

- Writing the operation's row to `workload_events` happens after the timer stops and is **excluded**.

An operation counts as failed if it raises any exception, including a legitimate business rejection such as an insufficient balance; the message is stored with the event.

6.5 Reproducibility

The sequence of operation types is drawn from a generator seeded with the configuration's seed, and each operation's account and amount choices are drawn from a generator seeded with the seed plus the operation index. Two runs with the same configuration therefore issue the same logical operations in the same order. Timing, the interleaving of threads and the merchant chosen for a payment (selected in SQL with `ORDER BY RANDOM()`) are not deterministic.

7. Telemetry Collection

7.1 Sampling

When a run starts, the `TelemetryCollector` stores an INITIAL snapshot, then starts a daemon thread that stores a SAMPLE snapshot at a fixed interval (one second by default) until the run completes, and finally stores a FINAL snapshot. If a snapshot fails, the error is logged and counted in `telemetry_error_count`; the workload continues. Table 8 lists what one snapshot contains.

Table 8. Contents of one telemetry snapshot.

Source	Metrics	Notes
`pg_stat_activity`	Active, idle, waiting and lock-blocked sessions	Waiting means any wait event, including idle client waits; only sessions with wait type Lock are counted as blocked. Sessions of all databases on the server are counted.
`pg_stat_database`	Commits, rollbacks, blocks read and hit, tuples inserted, updated and deleted, deadlocks	Cumulative counters for the current database.
`pg_locks`	Total locks; locks not yet granted	The count of ungranted locks is the lock-wait metric.
`pg_database_size()`, `max_connections`	Database size in bytes; configured connection limit	
`psutil`	Host CPU percent; host memory percent	Whole-machine values, not PostgreSQL alone.

The cache hit ratio is derived from the block counters:

$$\text{cache hit ratio} = 100 \times \text{blks_hit} \,/\, (\text{blks_hit} + \text{blks_read})$$

7.2 Experiment summary

After the run, the collector writes one row to `experiment_summaries`: the error rate and latency percentiles from the workload result; the maximum of active connections, blocked sessions and ungranted locks over all snapshots; the differences between the FINAL and INITIAL commit, rollback and deadlock counters; the average host CPU and memory over all snapshots; the database size before and after; and the number of snapshots and errors.

7.3 Interpreting the telemetry

- **Host metrics are machine-wide.** CPU and memory include the workload generator, the API, Docker and every other process on the machine. A model trained on them is specific to the machine and to what else it was doing.
- **The first CPU reading is not meaningful.** `psutil.cpu_percent(None)` reports usage since the previous call and returns 0.0 on its first call [14]. The INITIAL snapshot is included in the CPU average, which biases the average slightly downwards.
- **The cache hit ratio is cumulative.** It is computed from counters accumulated since the server's statistics were last reset, taken at the FINAL snapshot, not from the blocks touched by the experiment alone.
- **Counter deltas include the platform's own activity.** Event inserts and telemetry inserts are commits too. The rollback delta is not a count of failed transactions: SQLAlchemy ends a connection block that was not committed with ROLLBACK, so every read-only operation contributes rollbacks. This is consistent with the recorded experiment in Section 13.3, which shows 159 rollbacks with an error rate of 0%.
- **Sampling is coarse.** With one-second snapshots, a lock wait that lasts a few milliseconds will usually fall between samples.

8. Machine Learning Pipeline

8.1 Dataset

`ml/dataset.py` joins `experiment_runs` with `experiment_summaries` and expands the JSONB configuration into columns. Each completed experiment becomes one row. A **configuration key** is built from the scenario, concurrency, target TPS, duration, burstiness and the seven mix ratios; it identifies experiments that share a configuration regardless of seed.

8.2 Features and targets

Table 9 separates the model inputs, all of which are known before a workload runs, from the outputs, which are known only afterwards.

Table 9. Model inputs and outputs.

Role	Columns	Known when
Categorical feature	`scenario` (one-hot encoded; unknown categories are ignored at prediction time)	Before execution
Numeric features (11)	`concurrency`, `target_tps`, `duration_seconds`, `burstiness`, and seven operation-mix ratios	Before execution
Targets (5)	`average_latency_ms`, `p95_latency_ms`, `actual_tps`, `host_cpu_avg_percent`, `lock_wait_count_max`	Only after execution

The preprocessing step is a scikit-learn `ColumnTransformer` that lists the feature columns explicitly and is created with `remainder="drop"`. Any other column of the dataset, including every measured quantity, is discarded before a model sees the data. Leakage of post-execution information into the features [11] is therefore prevented by construction, and a unit test asserts that no target column appears in the feature set.

8.3 Training and model selection

- **Gate.** Training is refused unless there are at least six usable experiments, four distinct configurations and two scenarios. A usable experiment is a completed one with all five targets present.
- **Split.** A single `GroupShuffleSplit` holds out 25% of the configurations (random state 42). Grouping by configuration key keeps repeated runs of one configuration on the same side of the split.
- **Candidates.** For each target, three pipelines are fitted on the training fold: linear regression, a random forest of 100 trees and gradient boosting with default settings (random state 42 for both ensembles).
- **Selection.** The candidate with the lowest mean absolute error (MAE) on the held-out fold is kept. Root mean squared error (RMSE) and the coefficient of determination (R^2) are recorded as well.
- **Persistence.** The selected pipeline is saved with joblib, next to a JSON file that records the algorithm, the sample counts, the metrics, the scenarios seen and the minimum and maximum of every numeric feature.

The persisted model is the one fitted on the training fold; it is not refitted on all available data after selection. With a larger dataset, k-fold cross-validation grouped by configuration and a final refit would be preferable to a single split.

8.4 Prediction and out-of-range warnings

`predict_performance(workload)` builds one feature row from a workload description, filling the mix ratios from the scenario's default mix when none is given, and applies each persisted model. It then compares the request with the recorded training metadata and returns a warning for a scenario that was not in the dataset and for each numeric feature that lies outside the recorded minimum and maximum, for example `concurrency=50 is outside the training range 4.0–12.0`. If no model artefacts exist, prediction fails with an explicit error instead of returning a default value; the API reports this as HTTP 409.

9. What-If Analysis

The what-if engine answers questions of the form “what does the model expect if concurrency is doubled?” without running a workload. `compare_workloads(baseline, modified)` normalizes both configurations, predicts each, and reports for every metric the baseline value, the modified value, the absolute change and the percentage change:

$$\text{percentage change} = 100 \times (\text{modified} - \text{baseline}) \,/\, |\text{baseline}|$$

The percentage change is undefined, and reported as null, when the baseline prediction is zero. The comparison is labelled HIGH_IMPACT if any metric changes by at least 20%, MEDIUM_IMPACT if any changes by at least 5%, and LOW_IMPACT otherwise. Warnings from both predictions are merged and returned with the result.

`sweep_workload` predicts a series of configurations that differ in one parameter (concurrency or target TPS), and `sensitivity_analysis` reduces a sweep to one number per metric: the change in the prediction between the last and the first point divided by the change in the parameter. The comparison is available through the API and the web interface; sweeps and sensitivity are available through `scripts/what_if.py`, with JSON and CSV export.

These outputs describe the trained model, not the database. A sensitivity value states how the model's prediction responds to a parameter; it is not evidence of a causal effect in PostgreSQL, and it is only as reliable as the model in the region being explored.

10. Optimization Advisor

The advisor inspects the measured metrics of one experiment and applies six threshold rules (Table 10). The thresholds are fields of a `Thresholds` dataclass and can be overridden. Each rule that fires produces a recommendation with an identifier, category, severity, description, the evidence values that triggered it, a suggested investigation, the expected benefit and a flag stating that the recommendation requires validation. Recommendations are sorted by severity. If no rule fires, the status is NORMAL.

Table 10. Rules of the optimization advisor.

Category	Condition	Severity
CPU	Average host CPU ≥ 80%	HIGH
MEMORY	Average host memory ≥ 85%	HIGH
CONCURRENCY	Maximum active connections ÷ `max_connections` ≥ 0.8	HIGH
LOCKING	Maximum count of ungranted locks > 0	HIGH
LATENCY	P95 latency ÷ average latency ≥ 2.0	MEDIUM
QUERY	Cache hit ratio < 95%	MEDIUM

The advisor is strictly advisory: it never alters PostgreSQL configuration, creates an index or rewrites a query. It also attempts to read the twenty most expensive statements from `pg_stat_statements`. When the extension is unavailable, as it is in the project's container, the query fails, the advisor continues with aggregate telemetry and attaches a warning that query-level analysis is unavailable. No query-level finding is produced without that evidence.

One rule is currently inactive for stored experiments. The connection rule needs `max_connections`, which is recorded in every telemetry sample but is not carried into `experiment_summaries`; `analyze_experiment` therefore never supplies it, and the rule can only fire when `analyze_metrics` is called directly with that value. Copying the value into the summary would activate it.

11. Validation Framework

11.1 Lifecycle

A validation tests one prediction against one real execution. Its lifecycle is shown in Figure 4. The record is created before anything else happens and moves through the states PENDING, RUNNING and COMPLETED or FAILED. Because the prediction is stored before the workload runs, it cannot be adjusted after the measurement is known. If prediction or execution raises an exception, the record is marked FAILED with the error message and no error metrics are computed.

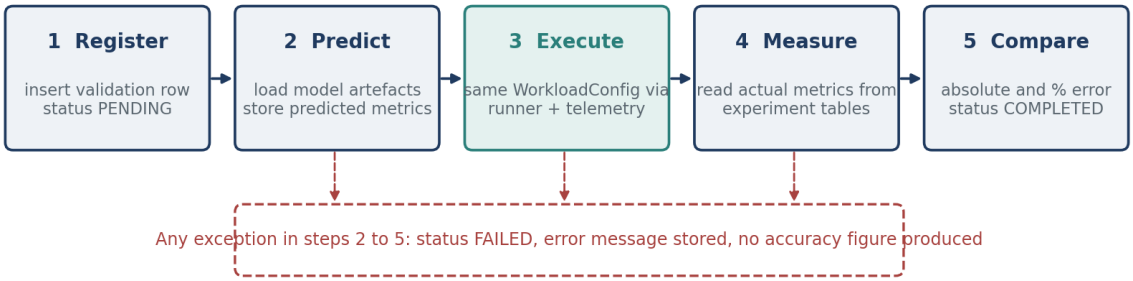


Figure 4. Lifecycle of a validation record as implemented in `run_validation`.

11.2 Error metrics

For a metric with predicted value $\hat{y}$ and measured value y, the per-run errors are:

$$\text{absolute error} = |\hat{y} - y| \qquad \text{percentage error} = 100 \times |\hat{y} - y| \,/\, |y|$$

The percentage error is null when the measured value is zero. Over a set of n completed validations, the summary reports, for each metric:

$$\text{MAE} = (1/n)\,\Sigma\,|\hat{y}_i - y_i| \qquad \text{RMSE} = \sqrt{(\,(1/n)\,\Sigma\,(\hat{y}_i - y_i)^2\,)}$$

$$R^2 = 1 - \Sigma\,(\hat{y}_i - y_i)^2 \,/\, \Sigma\,(y_i - \bar{y})^2 \qquad \text{MAPE} = (100/m)\,\Sigma\,|\hat{y}_i - y_i| \,/\, |y_i|$$

R^2 is null when the measured values have no variance, and MAPE is taken over the m runs whose measured value is non-zero. Summaries are produced overall and grouped by scenario, by concurrency and by target TPS. A separate script, `scripts/run_benchmark.py`, runs workloads with telemetry but without any model, providing measurements that do not depend on the ML pipeline.

12. API and User Interface

12.1 REST API

The FastAPI application (declared version 7.0.0) exposes twelve endpoints under `/api`, listed in Table 11, and serves the web interface from the same origin. Requests are validated by Pydantic models. Database errors are returned as HTTP 503, unknown identifiers as 404, invalid workloads as 422 and missing model artefacts as 409. The API has no authentication; it is intended for local use.

Table 11. REST endpoints.

Method and path	Purpose
GET `/api/health`	Checks the database connection.
GET `/api/database/summary`	Row counts of the six banking tables.
GET `/api/system/status`	Database, telemetry and model availability.
GET `/api/experiments`	Recent experiments with summary metrics (limit up to 500).
GET `/api/experiments/{id}`	One experiment with its telemetry samples and linked validations.
POST `/api/predictions`	Predicted metrics and warnings for a workload.
POST `/api/what-if`	Baseline-versus-modified comparison.
GET `/api/recommendations/{id}`	Optimization assessment of a stored experiment.
GET `/api/validation`	List of validation records.
GET `/api/validation/summary`	Aggregate error metrics over completed validations.
GET `/api/validation/{id}`	One validation record.
POST `/api/validation`	Runs a validation: predicts, executes the workload, compares.

`POST /api/validation` is synchronous: the HTTP request stays open for the whole duration of the workload. Because the handler is an ordinary function, FastAPI runs it in a worker thread and the server continues to answer other requests, but the caller must wait, and a long run can exceed a client timeout. The endpoint accepts scenario, concurrency, rate, duration, seed and telemetry interval; a custom operation mix is not accepted through this endpoint. There is no endpoint that starts a plain workload run without a validation; that is done with `scripts/run_workload.py`.

12.2 Web interface

The interface consists of `index.html`, `styles.css` and `app.js`, with no external libraries. It provides six views: Dashboard (system status, latest measured metrics, a performance history chart and recent experiments), Experiments (a filterable table with a detail drawer showing telemetry), Prediction, What-If Analysis, Optimization and Validation. Charts are drawn as inline SVG. A dark and a light theme are available and the choice is kept in the browser's local storage.

The interface was written so that it never displays a value the API did not return. Loading states are shown as skeletons, empty and error states are stated explicitly with the API's message, missing numbers are shown as a dash and not as zero, and predicted values are

labelled separately from measured values. Text received from the API is HTML-escaped before insertion.

13. Testing and Results

This section separates three kinds of evidence: automated tests, measurements recorded from real PostgreSQL runs, and model outputs. All figures are taken from files in the repository (`docs/PHASE*_RESULTS.md` and `artifacts/`). The measurements come from one development machine, a small number of ten-second runs and no repetitions; they demonstrate that the platform works and are not a statistical characterisation of PostgreSQL.

13.1 Automated tests

Table 12 lists the test modules, the number of test cases in each, and what they verify.

Table 12. Test modules and what each verifies.

Module	Cases	Needs PostgreSQL	What is verified
`test_phase1.py`	4	Yes	Tables exist and seed counts match; a successful transfer debits, credits and records one row; an insufficient-balance transfer changes nothing; health and summary endpoints respond.
`test_workload.py`	5	No	Required scenarios exist; default mix and seed; mixes that do not sum to one are rejected; non-positive concurrency, rate and duration are rejected.
`test_telemetry.py`	2	No	Percentile interpolation and average; counter delta with missing values.
`test_ml.py`	2	No	Feature set contains no target column; prediction without model artefacts raises an error.
`test_what_if.py`	4	No	Comparison deltas, sweep and sensitivity, input validation and warning propagation, using a stub predictor.
`test_optimization.py`	3	No	CPU and tail-latency rules fire; healthy metrics give no recommendation; lock rule carries its evidence.
`test_validation.py`	3	No	Absolute and percentage error; zero and missing actual values; summary by scenario.
`test_frontend.py`	1	No	The page and its two assets are served.

The suite contains 24 test cases in eight modules. For this report it was re-run on 9 October 2026 with Python 3.11.9 and pytest 8.4.2 while the PostgreSQL container was stopped: the 20 cases that do not need the database passed, and the four integration cases in `test_phase1.py` were not executed in that run. The project's phase records show those four cases passing in the full-suite runs made while the database was available.

The tests cover the pure logic of each module and the transfer transaction. They do not cover concurrent transfers, the pacing loop of the runner, the telemetry collector against a live server, `train_models`, or the prediction, what-if and validation endpoints.

13.2 Workload runs

Table 13 lists the six workload runs recorded after the generator was completed. Every run executed for ten seconds, completed all requested operations with no failures, and achieved the target rate.

Table 13. Recorded workload runs (source: `docs/PHASE2_RESULTS.md`).

Scenario	Concurrency	Target TPS	Operations	Actual TPS	Average latency (ms)
NORMAL_DAY	4	10	100	10.09	9.23
NORMAL_DAY	8	20	200	20.07	9.33
SALARY_DAY	4	10	100	10.09	10.55
FESTIVAL_SPIKE	8	20	200	20.06	10.67
HIGH_CONCURRENCY_ TRANSFER	12	30	300	30.06	11.84
LOAN_PROCESSING	4	15	150	15.09	6.83

13.3 Telemetry for one experiment

Table 14 shows the summary of one NORMAL_DAY experiment at concurrency 4 and 10 TPS. Eleven snapshots were stored with no telemetry errors. No session was blocked on a lock and no deadlock occurred. The maximum number of simultaneously active connections observed was one, which shows how lightly the database was loaded.

Table 14. Telemetry summary of one NORMAL_DAY experiment (sources: `docs/PHASE3_RESULTS.md`, `docs/PHASE6_RESULTS.md`).

Metric	Value	Metric	Value
Actual TPS	10.08	Commit delta	461
Average latency	9.84 ms	Rollback delta	159
P50 latency	9.26 ms	Deadlock delta	0
P95 latency	13.31 ms	Cache hit ratio	99.69%
P99 latency	23.01 ms	Average host CPU	8.07%
Error rate	0%	Average host memory	92.07%
Max. active connections	1	Database size before	10,730,519 bytes
Max. ungranted locks	0	Database size after	10,828,823 bytes

13.4 Model training

At training time the database held eight completed experiments, seven of them usable, covering seven configurations and five scenarios, with 77 telemetry samples in total. The grouped split placed five experiments in the training fold and two in the test fold. Table 15 gives the selected model and its held-out metrics for each target, as stored in `artifacts/models/*.json`.

Table 15. Selected model and held-out metrics per target (5 training samples, 2 test samples).

Target	Selected model	MAE	RMSE	R^2
Average latency (ms)	Gradient boosting	2.516	2.733	−0.189
P95 latency (ms)	Linear regression	1.036	1.132	0.265
Actual TPS	Linear regression	0.355	0.495	0.996
Host CPU (%)	Gradient boosting	2.660	3.376	0.594
Max. lock waits	Linear regression	0.000	0.000	1.000

These numbers must be read with the sample size in mind. Each metric is computed from two test points, so none of them is statistically meaningful.

- **Average latency.** A negative R^2 means the model predicted the two test points worse than their mean would have. Reproducing the predictions from the stored artefacts shows that the two held-out experiments were the HIGH_CONCURRENCY_TRANSFER and LOAN_PROCESSING runs, whose scenarios do not occur in the training fold at all.
- **Actual TPS.** The high R^2 reflects the fact that every run achieved its target rate, so actual TPS is almost equal to the input feature `target_tps`. It shows that the model learned an identity, not that it can predict saturation.
- **Lock waits.** Every experiment recorded zero lock waits, so a constant prediction of zero is trivially perfect. The model has learned nothing about contention.
- **Dimensionality.** After one-hot encoding, each model receives 14 inputs but is fitted on 5 rows. A linear model with more inputs than rows fits its training data exactly and its coefficients are not uniquely determined; the mix ratios are also a deterministic function of the scenario, and duration and burstiness are constant in the dataset.

13.5 What-if analysis

The what-if engine was exercised on a NORMAL_DAY baseline at concurrency 50 and 100 TPS. Table 16 gives the comparison when concurrency is raised to 100, and Figure 5 shows two parameter sweeps. Every configuration in this exercise lies outside the training range (concurrency 4 to 12, target TPS 10 to 30), and the engine attached the corresponding warnings to every result. The outputs were reproduced from the stored model artefacts while preparing this report and match the recorded values.

Table 16. What-if comparison: concurrency 50 to 100 at 100 TPS (model predictions; source: `docs/PHASE5_RESULTS.md`).

Predicted metric	Baseline	Modified	Change
Average latency (ms)	9.33	9.33	0.00%
P95 latency (ms)	16.63	17.26	+3.78%
Actual TPS	103.38	120.59	+16.65%
Host CPU (%)	22.25	22.25	0.00%
Max. lock waits	0	0	undefined (zero baseline)

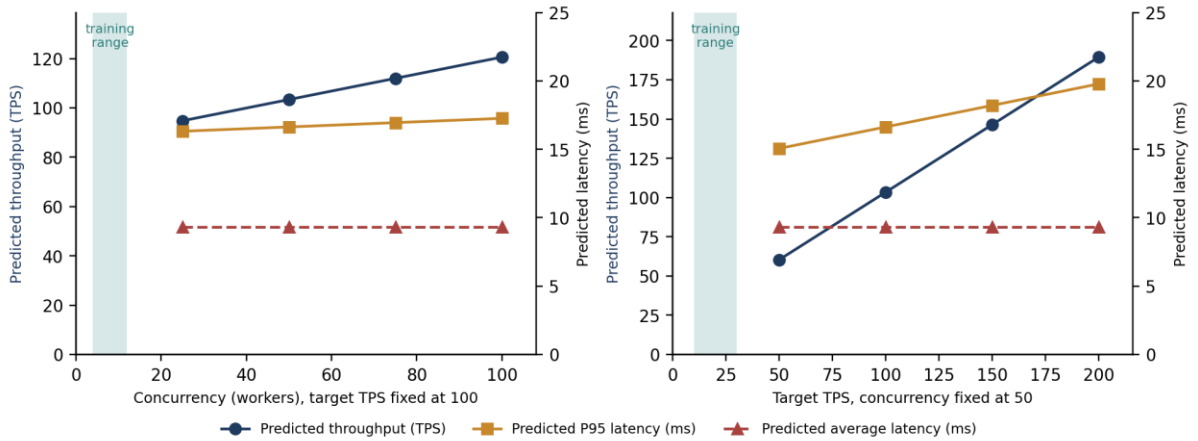


Figure 5. Predicted metrics for a concurrency sweep (left) and a target-TPS sweep (right), from `artifacts/concurrency-sweep.csv` and `artifacts/tps-sweep.csv`. The shaded band is the range covered by the training data; all swept points lie outside it.

The result is labelled MEDIUM_IMPACT because the largest change is 16.65%. The table and the sweeps illustrate the two failure modes described in Section 2.4. The gradient-boosted models for average latency and CPU return the same value at every point, because a tree ensemble cannot extrapolate; the value 9.33 ms is exactly the latency of one training experiment. The linear models for TPS and P95 extend a straight line, which produces a predicted throughput of 120.6 TPS for a workload whose arrival rate is capped at 100 TPS. Such a result is physically impossible for the runner and is a clear sign that the model is being used outside the region where it is valid. The exercise demonstrates that the mechanism and its warnings work; its numbers should not be used for capacity planning.

13.6 Optimization assessment

The advisor was run on the experiment of Table 14. It returned status ELEVATED with one HIGH finding, potential memory pressure, because host memory averaged 92.07% against the 85% threshold. This is a property of the development machine and not of PostgreSQL, which illustrates the limitation of host-level metrics. The CPU, lock, cache and tail-latency rules did not fire (P95 ÷ average = 1.35). The advisor reported that query-level analysis was unavailable because `pg_stat_statements` could not be read.

13.7 Predicted versus actual

One validation has been completed and stored (`artifacts/validation.json`): NORMAL_DAY, concurrency 10, 10 TPS, 10 seconds, seed 42. The configuration lies inside the training range and the prediction carried no warnings. Table 17 and Figure 6 compare the stored prediction with the subsequent measurement.

Table 17. Result of the recorded validation run.

Metric	Predicted	Actual	Absolute error	Percentage error
Average latency (ms)	9.46	18.83	9.37	49.75%
P95 latency (ms)	13.30	39.73	26.43	66.52%
Throughput (TPS)	12.15	10.08	2.07	20.57%
Host CPU (%)	13.13	26.55	13.42	50.54%
Max. lock waits	0	0	0	undefined (actual is zero)

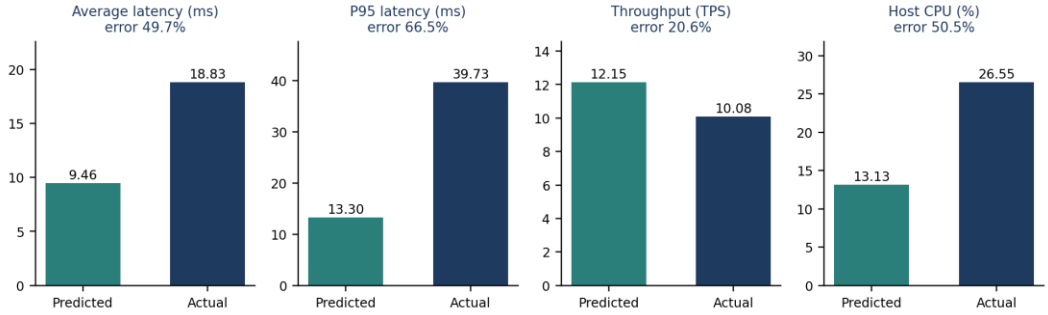


Figure 6. Predicted and measured values of the recorded validation run, from `artifacts/validation.json`.

The prediction was substantially wrong. Measured latency was about twice the predicted value, measured CPU about twice the predicted value, and the model predicted a throughput above the 10 TPS at which operations were submitted. One validation run cannot establish the accuracy of the models in general; it does show that they are not yet usable for prediction.

13.8 Benchmark runs and run-to-run variation

Two model-independent benchmark runs were recorded about one minute before the validation run (`artifacts/benchmark.csv`), both at concurrency 10 and 10 TPS with all 100 operations successful: NORMAL_DAY with an average latency of 21.41 ms and P95 of 42.04 ms, and SALARY_DAY with an average of 25.37 ms and P95 of 42.00 ms. Figure 7 places these next to the earlier runs.

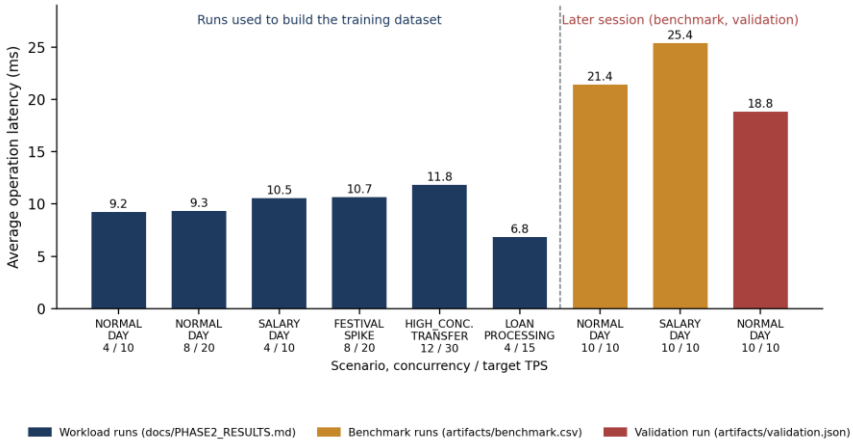


Figure 7. Average operation latency of all recorded runs. The three runs of the later session are about twice as slow as the earlier runs at comparable load.

The three runs of the later session, including the validation run, show roughly double the latency of the earlier runs, although the offered load was similar. The benchmark runs did not involve the model, so the difference is not a modelling artefact: the machine itself was slower in the later session. The repository does not record the cause. Because CPU and memory are measured for the whole host, a plausible explanation is other activity on the development machine, but this was not verified. Whatever the cause, the observation matters for interpreting Table 17: a large part of the validation error reflects variation between sessions that no feature of the model describes, on top of the weakness of a five-sample training set.

14. Discussion

14.1 What has been achieved

The engineering objectives have been met. The schema enforces its invariants declaratively; the transfer transaction is atomic and uses ordered locking; the workload generator executes reproducible operation sequences at a controlled rate; telemetry is captured and linked to each run; the learning pipeline excludes leakage and refuses to train or predict without sufficient basis; what-if results carry range warnings; the advisor reports only what its evidence supports; and the validation engine ties each prediction to a measurement. The whole loop runs end to end from the command line, the API and the browser.

14.2 Why the models do not yet predict well

Three causes can be identified from the recorded data, and they are properties of the dataset more than of the pipeline.

The dataset is far too small. Five training rows cannot determine a model with fourteen inputs. Any of the three algorithms will reproduce the training points and behave arbitrarily elsewhere.

The experiments did not stress the database. An operation took about 10 ms, and the highest rate used was 30 TPS. The average number of operations in progress at any moment is the arrival rate multiplied by the service time, which is at most $30 \times 0.012 \approx 0.36$. The system was idle most of the time: the maximum number of active connections observed in the recorded experiment was one, and lock waits were always zero. In this regime latency is nearly independent of concurrency and rate, so the features that the model is asked to learn from carry almost no signal, and the phenomena of real interest (queueing, lock contention and saturation) never appear in the data.

Uncontrolled variation is larger than the effects being modelled. The difference in latency between sessions (Section 13.8) is larger than any difference between scenarios within a session. Without repeated runs, the noise level cannot even be estimated, and without features describing the state of the host, it cannot be explained.

14.3 Value of the result

The validation result is negative, and reporting it as such is the purpose of the validation framework. A platform that only ever confirmed its own predictions would be untrustworthy.

The framework detected that the models are inadequate, the stored evidence is sufficient to diagnose why, and the same tooling can generate the larger and more demanding dataset that is needed. In that sense the project has delivered the instrument and a first, honest measurement with it.

15. Limitations

15.1 Data and models

- Seven usable experiments, five of them used for fitting; each held-out metric rests on two points; one completed validation run.
- All experiments lasted ten seconds with burstiness 0, used concurrency 4 to 12 and 10 to 30 TPS, and covered five of the seven scenarios. MONTH_END and CARD_PAYMENT_SPIKE were not measured.
- No experiment produced lock waits, deadlocks or saturation, so the lock-wait model is trivial and the throughput model has only seen runs that met their target.
- A single train/test split is used, the selected model is not refitted on all data, and predictions have no confidence interval.
- Models are specific to the machine on which the experiments were run.

15.2 Workload realism

- Accounts are chosen uniformly from the first 1,000 active accounts. Real workloads are skewed towards a small set of hot accounts, which is what produces lock contention.
- The burstiness parameter is stored but does not affect arrivals; arrivals are evenly spaced.
- The data set is small (about 10 MB), so it fits in memory and the cache hit ratio is always high.
- Scenarios differ only in their operation mix; none varies the arrival rate over time within a run.

15.3 Measurement

- Latency excludes time queued in the thread pool and includes an account-lookup query (Section 6.4).
- CPU and memory are host-wide; the first CPU reading is zero; the cache hit ratio is cumulative (Section 7.3).
- The connection pool limits database concurrency to fifteen connections.
- `pg_stat_statements` is not enabled, so there is no per-query evidence and no query-plan analysis.

15.4 Engineering

- The connection-pressure rule of the advisor is inactive for stored experiments (Section 10).
- The validation endpoint is synchronous and cannot accept a custom operation mix; sweeps are not exposed through the API.

- The API has no authentication and is intended for local use only.
- Only PostgreSQL is containerised; the application runs from a local Python environment. There is no continuous integration.
- Concurrent behaviour, the runner, the collector and model training have no automated tests.
- This report contains no screenshots of the web interface, because the database was not running when the report was prepared.

16. Future Work

- **Generate a dataset that covers the interesting region.** Run a grid over all seven scenarios, concurrency from 1 to beyond the connection limit, and rates up to and past saturation, with several repetitions per configuration and longer durations, so that queueing and lock contention appear in the data and run-to-run variance can be measured.
- **Make the workload capable of contention.** Add skewed (for example Zipfian) account selection, implement burstiness in the arrival process, and remove the per-operation account lookup from the measured path.
- **Improve measurement.** Record response time including queueing; measure the PostgreSQL container's CPU and memory instead of the host's; discard the first CPU reading; compute per-experiment cache statistics from counter deltas; enable `pg_stat_statements`.
- **Strengthen evaluation.** Use grouped k-fold cross-validation, compare against simple baselines, refit the chosen model on all data, and report prediction intervals.
- **Extend the advisor.** Carry `max_connections` into the summary to activate the connection rule, and add evidence from query statistics and `EXPLAIN (ANALYZE)` once available.
- **Harden the application.** Run validations as background jobs with status polling, expose sweeps through the API, containerise the application, and add concurrency tests and continuous integration.

17. Conclusion

FinTwin set out to build a system that can learn how a PostgreSQL banking database performs under different workloads and can check its own predictions. The platform that this requires has been built and integrated: a constrained relational schema, atomic and deadlock-aware transactions, a configurable workload generator, a telemetry collector, a leakage-safe learning pipeline, a what-if engine, a rule-based advisor, a validation engine, a REST API and a web interface, with automated tests for the logic of each part.

The predictive component is at the stage of a proof of mechanism. With seven lightly loaded experiments the models cannot generalise, and the recorded validation run, with errors between 20% and 67%, shows this plainly. The analysis in this report identifies the reasons, namely too few experiments, no load high enough to cause contention, and uncontrolled variation between sessions, and each has a direct remedy that the existing tooling can carry out. The principal contribution of the project is therefore the experimental platform and its

discipline of keeping predictions and measurements separate and comparable; the next step is to use it to collect the data that accurate prediction needs.

References

[1] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2019.

[2] M. Grieves and J. Vickers, “Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems,” in Transdisciplinary Perspectives on Complex Systems, Cham: Springer, 2017, pp. 85–113.

[3] D. Van Aken, A. Pavlo, G. J. Gordon and B. Zhang, “Automatic Database Management System Tuning Through Large-scale Machine Learning,” in Proc. ACM SIGMOD International Conference on Management of Data, 2017, pp. 1009–1024.

[4] D. E. Difallah, A. Pavlo, C. Curino and P. Cudré-Mauroux, “OLTP-Bench: An Extensible Testbed for Benchmarking Relational Databases,” Proceedings of the VLDB Endowment, vol. 7, no. 4, pp. 277–288, 2013.

[5] Transaction Processing Performance Council, TPC Benchmark C Standard Specification, Revision 5.11, 2010.

[6] E. G. Coffman, M. J. Elphick and A. Shoshani, “System Deadlocks,” ACM Computing Surveys, vol. 3, no. 2, pp. 67–78, 1971.

[7] The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: “Monitoring Database Activity”, “Explicit Locking” and “pg_stat_statements”. Available: https://www.postgresql.org/docs/16/

[8] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[9] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[10] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[11] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, “Leakage in Data Mining: Formulation, Detection, and Avoidance,” ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, article 15, 2012.

[12] FastAPI documentation. Available: https://fastapi.tiangolo.com/

[13] SQLAlchemy 2.0 documentation, “Connection Pooling”. Available: https://docs.sqlalchemy.org/en/20/core/pooling.html

[14] psutil documentation, “psutil.cpu_percent”. Available: https://psutil.readthedocs.io/

Appendix A. Reproducing the Experiments

The commands below are for Windows PowerShell from the project root and assume Docker and Python 3.11 or later. They are taken from the project's README.

```
Copy-Item .env.example .env
docker compose up -d
python -m venv .venv ; .\.venv\Scripts\Activate.ps1
python -m pip install -r backend/requirements.txt
$env:PYTHONPATH = (Get-Location).Path
python scripts/initialize_database.py
python scripts/seed_database.py
```

Run a workload with telemetry, inspect the dataset and train the models:

```
python scripts/run_workload.py --scenario NORMAL_DAY --concurrency 4 `
    --target-tps 10 --duration 10
python scripts/diagnose_ml_dataset.py
python scripts/train_models.py
```

Predict, compare, assess and validate:

```
python scripts/predict_workload.py --scenario NORMAL_DAY --concurrency 10 --target-tps 10
python scripts/what_if.py --concurrency 50 --target-tps 100 --change-concurrency 100
python scripts/what_if.py --concurrency 50 --target-tps 100 `
    --sweep-concurrency 25 50 75 100
python scripts/analyze_performance.py --experiment-id <UUID>
python scripts/validate_prediction.py --scenario NORMAL_DAY --concurrency 10 `
    --target-tps 10
python scripts/run_benchmark.py --scenarios NORMAL_DAY SALARY_DAY `
    --format csv --output artifacts/benchmark.csv
```

Start the API and web interface, and run the tests:

```
uvicorn backend.app.main:app --reload   # then open http://127.0.0.1:8000/
pytest
```